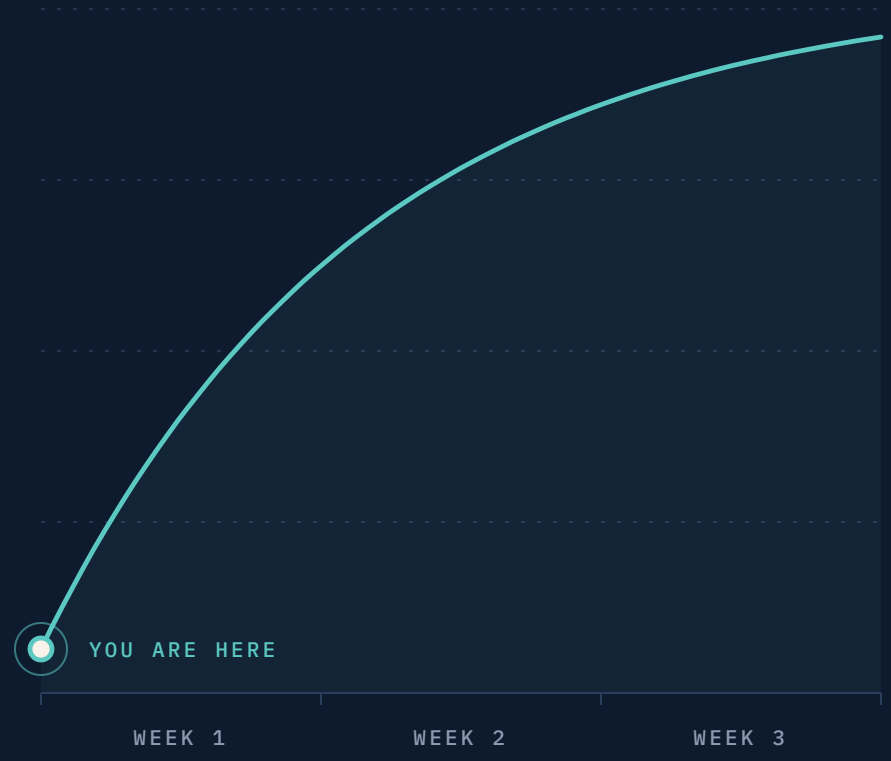
WEEK 1 · DAY 1 · OVERVIEW

APIs, Keys, and Secrets

From your first line of code to your own AI agents.
Every lesson is a fixed income problem.



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Welcome to Course 3

From today the morning numbers come from FRED, pulled by code. FRED wants a key on every request, in your name. Before any code touches a real key, show me what FRED sends back, checked against the Treasury file we trust. Get your own key and give it a home outside every file, notebook, and commit. Write the one function that finds it, tested. Pull the 10-year live and tie it to the snapshot. Then show me a key leaking, and the fix.

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 1

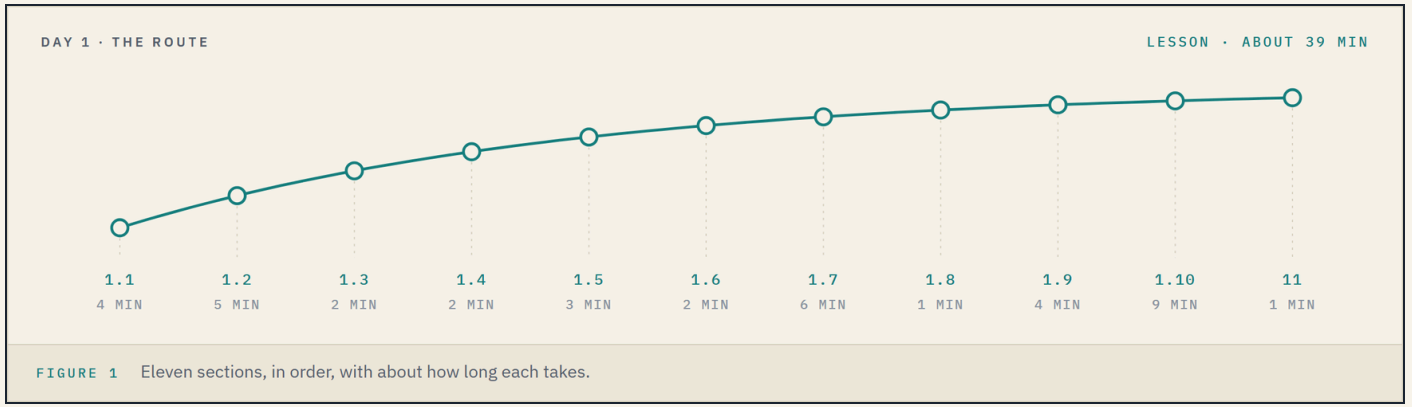


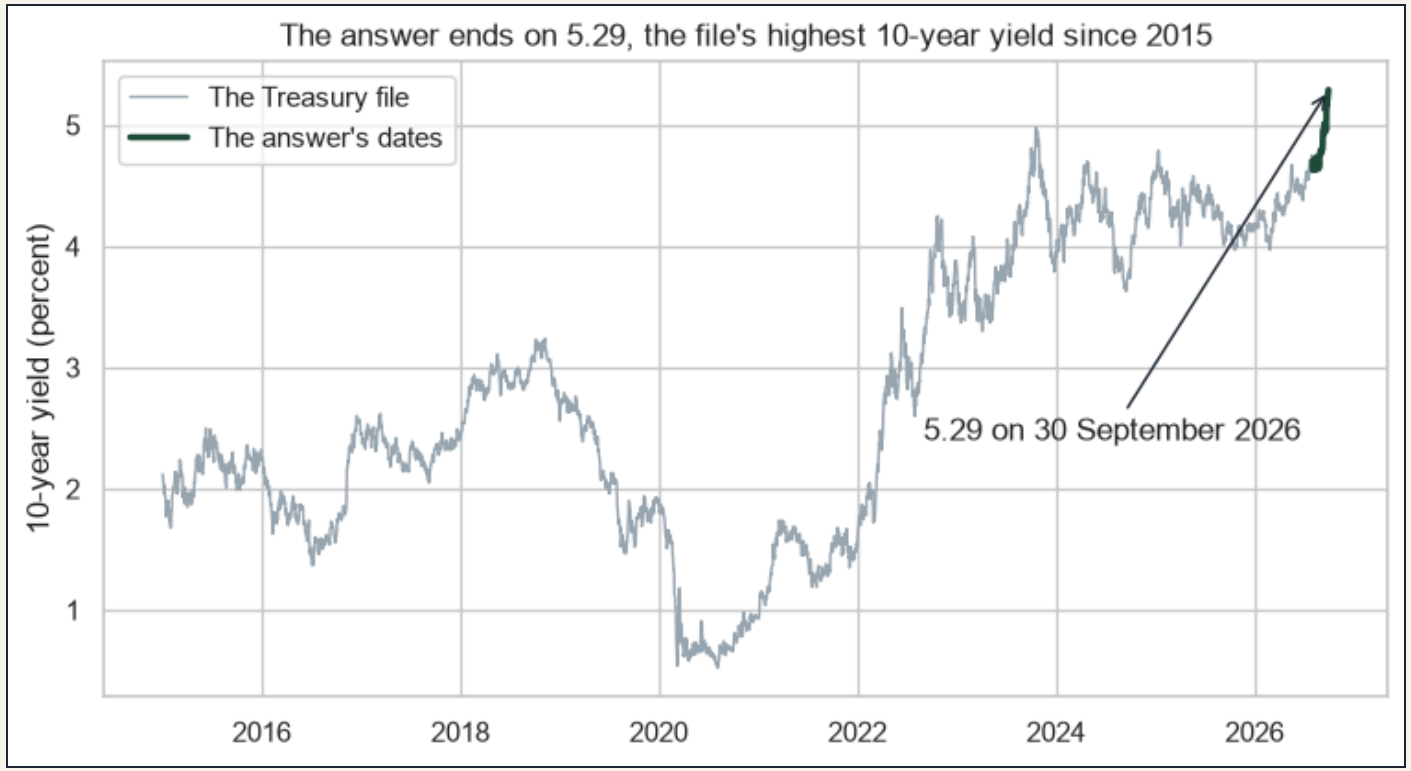
FIGURE 1 Eleven sections, in order, with about how long each takes.

	SECTION	ABOUT
1.1	What FRED Sends Back	4 min
1.2	The Request, Taken Apart	5 min
1.3	Your Own Key, Kept Like a Password	2 min
1.4	Environment Variables	2 min
1.5	The `.env` File and `.env.example`	3 min
1.6	`.gitignore` Before the First Commit	2 min

	SECTION	ABOUT
1.7	`get_secret`, Docstring First	6 min
1.8	Colab's Secrets Panel	1 min
1.9	The Key Cell and the First Live Pull	4 min
1.10	When a Key Leaks	9 min
11	Bring It Home	1 min

Time. About 39 minutes for the lesson, then about 12 for the four exercises.

1.1 What FRED Sends Back



AN ANSWER AS TEXT


```
{
  "realtime_start": "2026-10-02",
  "realtime_end": "2026-10-02",
  "observation_start": "2026-08-03",
  "observation_end": "2026-09-30",
  "units": "lin",
  "output_type": 1,
  "file_type": "json",
  "order_by": "observation_date",
  "sort_order": "asc",
  "count": 43,
  "offset": 0,
  "limit": 100000,
  "observations": [
```


FRED's answer is text you can read, and its 5.29 on 30 September is the Treasury file's 5.29.

1.2 The Request, Taken Apart



FIGURE 2 The request URL in five blocks, with the api_key block marked, and what comes back: status 200 and the body.

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Encode text for a URL	`=ENCODEURL("a b&c=d/e?f")`	`a%20b%26c%3Dd%2Fe%3Ff`

THE SAME ENCODING IN PYTHON

```
print(quote('a b&c=d/e?f', safe=''))
```

```
a%20b%26c%3Dd%2Fe%3Ff
```

The key rides inside the URL, so anything that prints a URL prints your key; FRED's error messages never repeat it.

1.3 Your Own Key, Kept Like a Password

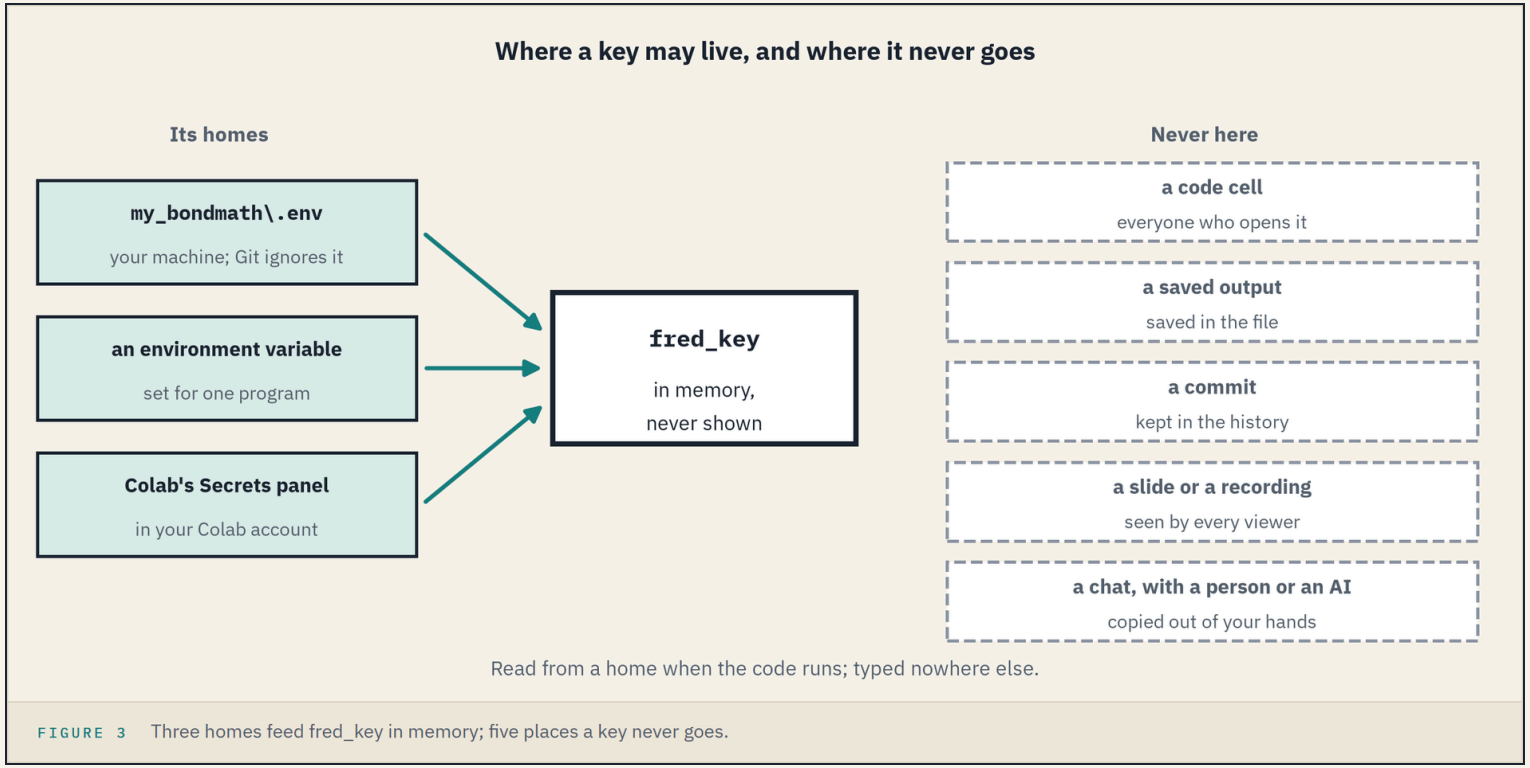


FIGURE 3 Three homes feed fred_key in memory; five places a key never goes.

EXCEL SIDE BY SIDE

JOB	RESULT
A key typed into a cell	still in the file, in `xl/sharedStrings.xml`, and back in B1 on reopening

A key is a password that works on every request: yours alone, and never typed where someone else can read it.

1.4 Environment Variables

IS IT THERE?

```
print('DEMO_API_KEY' in os.environ)
print(os.environ.get('DEMO_API_KEY'))
```

```
False
None
```

SET IT, COMPARE, REMOVE IT

```
os.environ['DEMO_API_KEY'] = FAKE_KEY
print('The value is the fake key:', os.environ['DEMO_API_KEY'] == FAKE_KEY)
del os.environ['DEMO_API_KEY']
print('DEMO_API_KEY' in os.environ)
```

```
The value is the fake key: True
False
```

IN POWERSHELL

```
# this terminal window only
$env:DEMO_API_KEY = "..."
# saved for your Windows user
[Environment]::SetEnvironmentVariable("DEMO_API_KEY", "...", "User")
```

A secret in the environment sits outside the code: the code asks for it by name.

1.5 The .env File and .env.example

THE DRILL'S .ENV

```
DEMO_API_KEY=fakefakefakefakefakefakefakefake
```

ITS .ENV.EXAMPLE

```
DEMO_API_KEY=
```

READ IT

```
from dotenv import dotenv_values
values = dotenv_values('.env')
print('Names:', list(values))
print('The value is the fake key:', values['DEMO_API_KEY'] == FAKE_KEY)
print('DEMO_API_KEY in os.environ:', 'DEMO_API_KEY' in os.environ)
```

```
Names: ['DEMO_API_KEY']
The value is the fake key: True
DEMO_API_KEY in os.environ: False
```

`.env` holds the value, `.env.example` holds the names, and only the names will be committed.

1.6 .gitignore Before the First Commit

THE .GITIGNORE

```
.env
cache/
*.db
__pycache__/
.pytest_cache/
```

WHICH LINE KEEPS IT OUT

```
!git -C "{work}" check-ignore -v .env
.gitignore:2:.env       .env
```

WHAT GIT SEES

```
!git -C "{work}" status --short --ignored
?? .env.example
?? .gitignore
!! .env
!! bondmath.py
!! course3_excel_reference.csv
!! course3_fred_format_dgs10.json
!! treasury_par_yields.csv
```

`.env` is ignored before anything is committed, and `check-ignore` names the line that does it.

1.7 get_secret, Docstring First

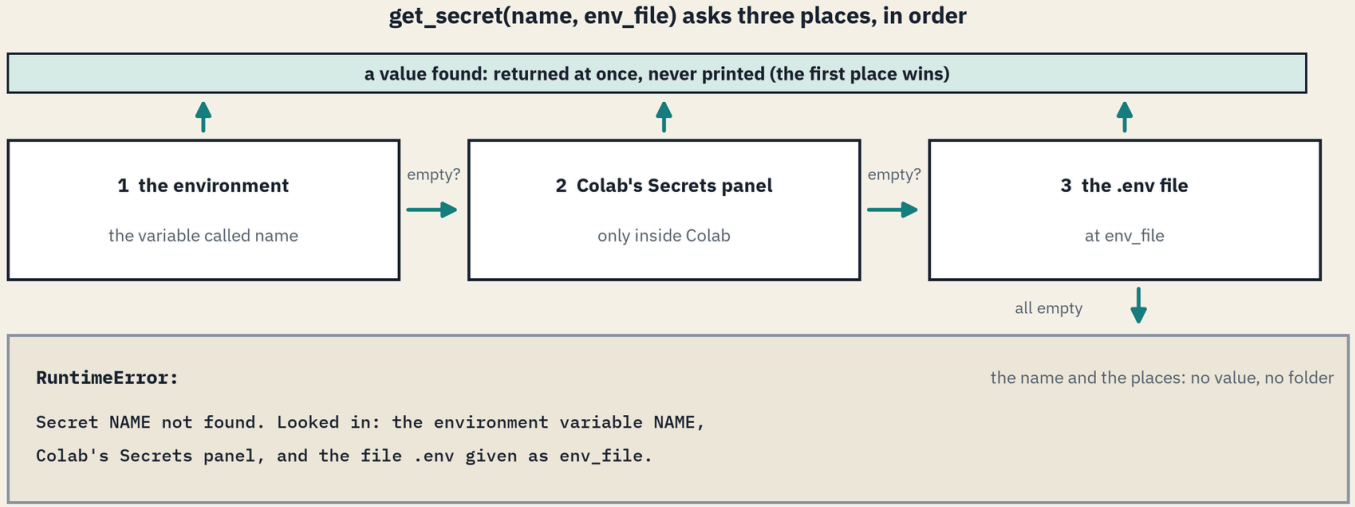


FIGURE 4 get_secret asks the environment, Colab's Secrets panel, then the .env file, and raises RuntimeError if all are empty.

FIND THE DRILL'S KEY

```
Found the fake key: True, 32 characters
```

FOUR TESTS

```
4 passed in 0.50s
```

`get_secret` **finds a secret in three places and never shows it,** `4 passed` **proves it, and the first commit holds four files and no** `.env`**.**

Keep your key in Colab's Secrets panel.

1.8 Colab's Secrets Panel

IN COLAB

- Open **Secrets**, the key icon on the left.
- Add `FRED_API_KEY` with your key as its value.
- Turn on **Notebook access**.

WHERE GET_SECRET ASKS COLAB

```
        from google.colab import userdata
        userdata = None
    if userdata is not None:
            value = userdata.get(name)
        except (userdata.SecretNotFoundError, userdata.NotebookAccessError):
```

In Colab the key lives in your account's Secrets panel, and `get_secret` asks there second.

1.9 The Key Cell and the First Live Pull

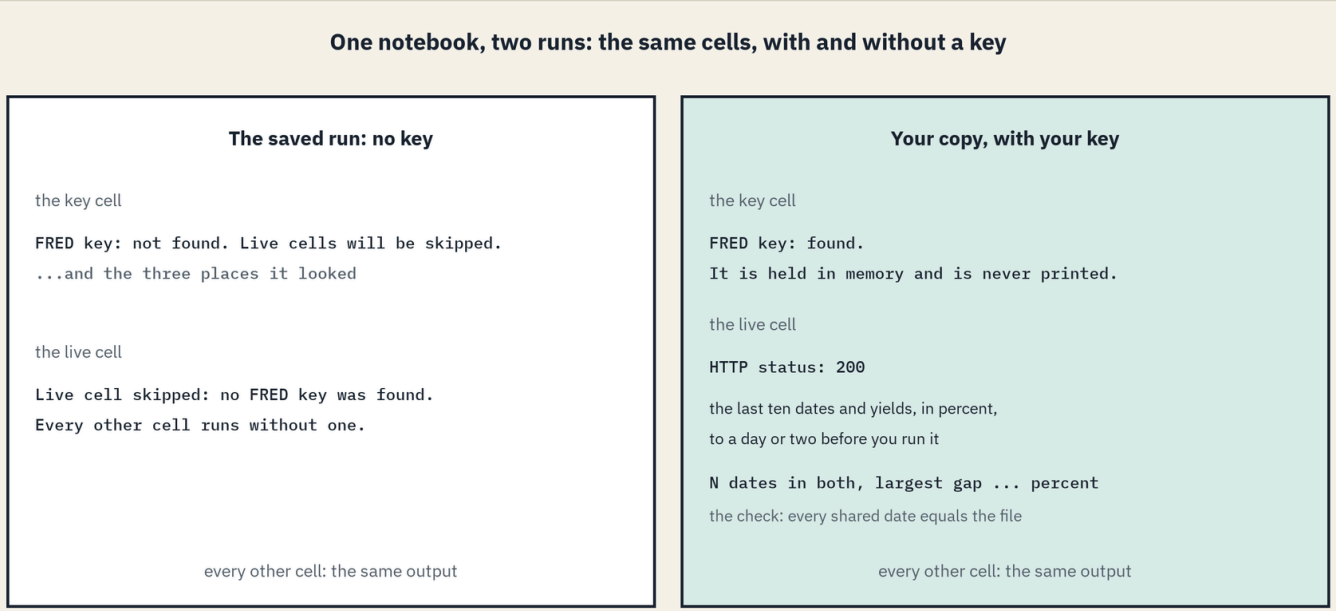


FIGURE 5 The same two cells run with no key, skipped, and with your key: status 200 and a tie to the snapshot.

EXCEL SIDE BY SIDE

JOB	RESULT
Public data from the web	header **observation_date**, **DGS10**; 16,913 rows, 1962-01-02 to 2026-10-29, on the day tested
WEBSERVICE on that address	**#VALUE!** when tested

The key cell and the live cell run for everyone: with no key they say so, with yours they pull FRED and tie it to the snapshot.

The Live Pull, Line by Line

THE KEY CELL

```
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
except RuntimeError as error:
    fred_key = None
```

THE LIVE CELL, ITS WORKING LINES

```
        live_response = requests.get(FRED_URL, params=live_params, timeout=30)
    print(f'HTTP status: {live_response.status_code}')
    live_common = live_obs.merge(treasury.reset_index()[['date', '10 Yr']], on='date')
    live_gap = (live_common['value'] - live_common['10 Yr']).abs().max()
    print(f'{len(live_common)} dates in both, largest gap {live_gap:.1e} percent')
```

The key cell and the live cell run for everyone: with no key they say so, with yours they pull FRED and tie it to the snapshot.

1.10 When a Key Leaks

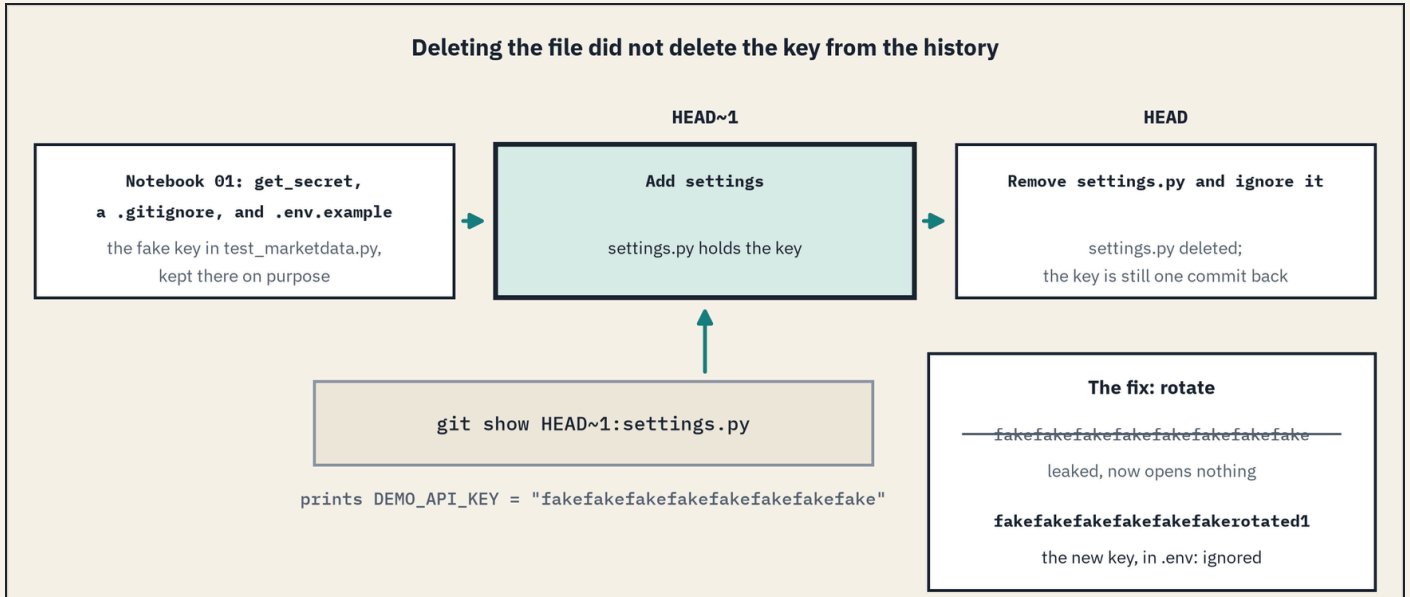


FIGURE 6 Add settings still holds the key after the delete; rotating leaves a new key in .env.

READ IT BACK

```
!git -C "{work}" show HEAD~1:settings.py

DEMO_API_KEY = "fakefakefakefakefakefakefakefake"
```

ROTATE

```
!! .env
get_secret returns the new key: True
```

Deleting a committed key leaves it in the history; rotating first, the old key removed, makes every copy worthless.

Give your real key its home.

Bring It Home

IN POWERSHELL

```
# the course's packages, if not yet installed
Set-Location "$HOME\projects\python-fixed-income"
python -m pip install -r course03_market_data\requirements.txt
# ignore .env before it exists
Set-Location "$HOME\projects\my_bondmath"
Add-Content .gitignore ".env", "cache/", "*.db"
Set-Content .env.example "FRED_API_KEY="
code .env
```

pytest shows `4 passed`; `.env` is never in the `git add` line.

You Can Now

- Take a request apart: endpoint, parameters, status code, body.
- Get your own FRED key, and keep it like a password.
- Keep a secret in the environment or a `.env` file, and compare it instead of printing it.
- Put `.gitignore` in place before the first commit, and prove it with `git check-ignore -v`.
- Find a secret in three places with `get_secret`, tested with `monkeypatch` and `tmp_path`.
- Find a deleted key in Git's history, and rotate it.

NEXT
WEEK 1 · DAY 2

JSON and the FRED API

FRED's answer read as JSON, its missing day kept as NaN and never 0, three tested functions that pull a series without showing the key, and a live pull tied to the Treasury file.